

O'REILLY[®]

2.1 Running Your First Pre-Built Agent



Introduction to Pre-Built Agents

- **What are Pre-Built Agents?**
 - Agent implementations with minimal configuration
 - Handle tool selection and execution automatically
 - Foundation for building autonomous AI systems
- **Why Use Pre-Built Agents?**
 - Rapid prototyping and deployment
 - Battle-tested implementation patterns
 - Easy to extend and customize for specific use cases

Pre-Built Agents in LangChain

- **Overview**

- `create\_agent` is an agent builder function in LangChain
- Creates graph-based agent automatically, built on top of LangGraph
- Implements ReAct-style reasoning loop

- **Basic Usage**

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-5", temperature=0.1)
agent = create_agent(model, tools=[search, get_weather])
```

Agent Execution Flow

- **The ReAct Loop**
 - Agent receives input and analyzes task
 - Decides which tools to use (if any)
 - Executes tools and receives observations
 - Iterates until reaching final answer
- **Agent Invocation:** Use `.invoke()` to execute the agent



```
result = agent.invoke(  
    {"messages": [{"role": "user", "content": "..."}]})
```


Defining Tools for Pre-built Agent

- **Creating Tools**
 - Define functions with type hints and docstrings
 - Docstrings help the agent understand tool purpose
 - Automatically converted to tool schemas

```
def get_weather(location: str) -> str:  
    """Get current weather for a location."""  
    return f"Weather in {location}: Sunny, 72°F"
```

Configuring the Model

- **Model Configuration Parameters**
 - **model:** Specific model version to use
 - **temperature:** Controls randomness (0.0-1.0)
 - Low temperature ~ low randomness
 - **max_tokens:** Maximum response length
 - **timeout:** Request timeout in seconds

```
model = ChatOpenAI(  
    model="gpt-5",  
    temperature=0.1,  
    max_tokens=1000,  
    timeout=30  
)
```

Common Pitfall: Treating Pre-Built as Production Ready

Pre-built agents are great for prototyping but often lack production requirements like proper error handling, rate limiting, cost controls, observability.

Instead, use them to validate ideas quickly, then build proper infrastructure around them.

O'REILLY®

2.2 Implementing Structured Outputs with JSON and Pydantic



Why Structured Output

- **The Problem**

- LLMs naturally generate unstructured text
- Agents need predictable, parseable outputs
- Multi-step workflows require consistent data formats

- **Benefits of Structured Output**

- Seamless interaction between components and agents
- Reliable parsing and validation
- Type safety and error reduction
- Enables tool calling and function execution

JSON Schemas

- Standard for describing JSON structure
- Defines data types, required fields, and constraints
- Supported by most LLM providers

```
{
  "type": "object",
  "properties": {
    "name": {"type": "string"},
    "age": {"type": "integer"},
    "email": {"type": "string", "format": "email"}
  },
  "required": ["name", "email"]
}
```

Pydantic Models for Structured Output

- What is Pydantic?
 - Python library for data validation using type hints
 - Provides runtime validation and serialization
 - Can generate JSON schemas automatically

```
from pydantic import BaseModel, Field

class UserInfo(BaseModel):
    name: str = Field(description="User's full name")
    age: int = Field(gt=0, description="User's age")
    email: str = Field(description="User's email address")

# Usage
user = UserInfo(name="John Doe", age=30,
email="john@example.com")
```

Structured Output with Langchain

```
from langchain_openai import ChatOpenAI
from pydantic import BaseModel, Field

class UserInfo(BaseModel):
    name: str = Field(description="User's full name")
    age: int = Field(gt=0, description="User's age")
    email: str = Field(description="User's email address")

model = ChatOpenAI(model="gpt-5")
structured_model = model.with_structured_output(UserInfo)

result = structured_model.invoke(
    "Extract user info: John Doe, 30 years old, john@example.com"
)
# result.name = "John Doe", result.age = 30, result.email = "john@example.com"
```


O'REILLY®

2.3 Integrating External Tools into an Agent



How Agent Frameworks Enable Tools

- Frameworks enable easy tool integration through
 - Standardized tool interfaces for consistent integration
 - Automatic tool description generation for LLM understanding
 - Built-in orchestration between reasoning and tool execution
- **Practical Example: Web Search with Tavily**
 - Tavily Search demonstrates the pre-built tool integration pattern
 - Same approach applies to other tools (databases, APIs, custom functions)
- We will extend the pre-built agent from 2.1 with the Tavily tool

Integrating the Tavily Search Tool



```
from langchain_tavily import TavilySearch

# Instantiate the search tool
search_tool = TavilySearch(
    max_results=5,
    search_depth="advanced", # "basic" or "advanced"
    include_raw_content=False,
    include_images=False
)
```


Adding Tools to Your Agent

```
from langchain_openai import ChatOpenAI
from langchain_tavily import TavilySearch
from langgraph.prebuilt import create_react_agent

# Configure the model
model = ChatOpenAI(model="gpt-5", temperature=0.1)

# Create tool(s)
search_tool = TavilySearch(max_results=5)

# Create agent with tools
agent = create_react_agent(
    model,
    tools=[search_tool], # Pass list of tools
)

# Agent automatically decides when to use search
result = agent.invoke({
    "messages": [{"role": "user", "content": "What are the
latest developments in AI?"}]
})
```


Best Practices for Tool Integration

- **Tool Selection Guidelines**

- Start with one tool and test thoroughly
- Choose tools that provide clear value to your use case
- Consider API costs and rate limits

- **Error Handling**

- Tool calls may fail (network issues, API limits)
- Agent should gracefully handle tool failures
- Always test edge cases

- **Performance Considerations**

- More tools = slower decision making
- Keep tool count minimal per agent (3-5 tools recommended)
- Use clear, distinct tool descriptions

O'REILLY®

2.4 Building Simple Multi-Step LLM Workflows



Chains Revisited

- **Why Use Chains?**

- Many real-world tasks require multiple processing steps
- Break complex problems into manageable components
- Reusable, modular workflow design

- **Example Use Cases**

- RAG: Retrieve documents → Format context → Generate answer
- Report generation: Research → Outline → Write → Review
- Data processing: Extract → Transform → Analyze → Summarize

Chain Components and Design

- **Core Building Blocks**
 - **Prompt Templates:** Structure and format inputs
 - **LLM Calls:** Process and generate responses
 - **Output Parsers:** Extract structured data from responses
 - **Data Transformations:** Process intermediate results
- **Benefits of Modular Design**
 - Easy to test individual components
 - Reusable across different workflows
 - Clear separation of concerns
 - Simplified debugging

Example: Writing Improvement Chain



```
def writing_improvement_chain(topic):  
    # Step 1: Generate first draft  
    draft = llm_call(f"Write a paragraph about {topic}")  
  
    # Step 2: Critique the draft  
    critique = llm_call(f"What could be improved?\n{draft}")  
  
    # Step 3: Rewrite with improvements  
    final = llm_call(f"Rewrite:\n{draft}\n\nFix:\n{critique}")  
  
    return final
```

Example: RAG Chain

```
def retrieve_documents(query):  
    # retrieval code given a query  
    # .....  
    return documents  
  
def rag_chain(query):  
    # Step 1: Retrieve relevant context  
    context = retrieve_documents(query)  
  
    # Step 2: Generate answer with context  
    prompt = f"Answer based on  
context.\n\nContext:\n{context}\n\nQuestion: {query}"  
    answer = llm_call(prompt)  
    return answer
```

O'REILLY®

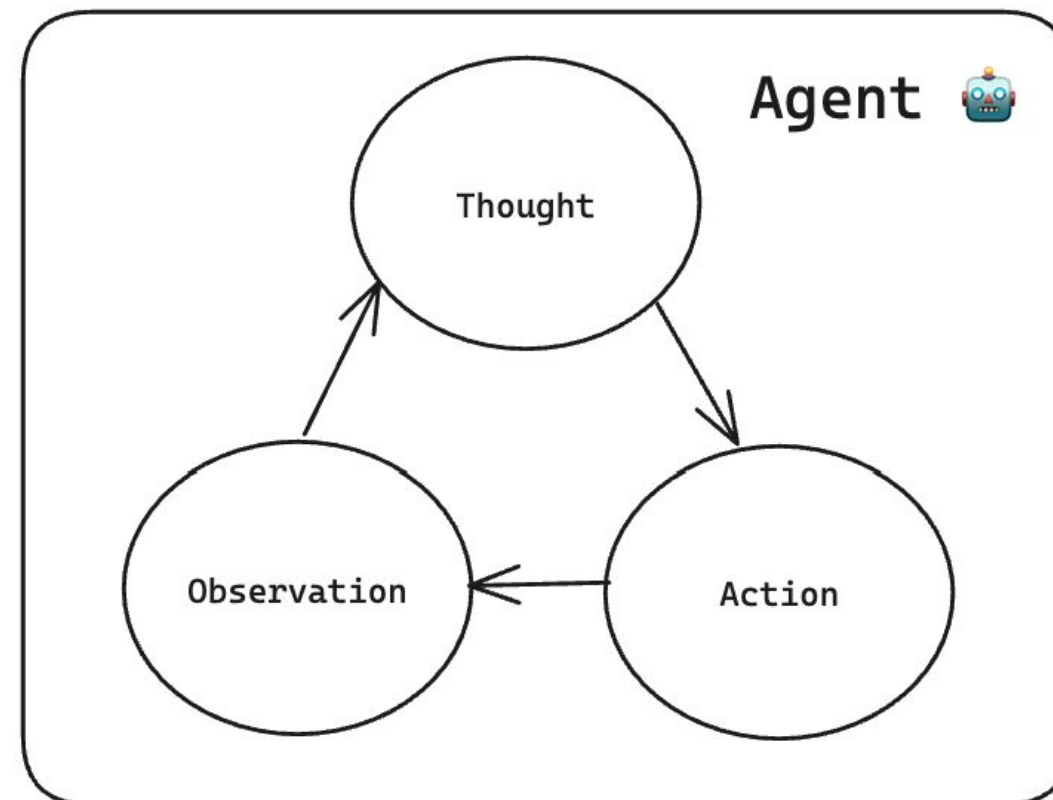
2.5 Create an Agent Class from Scratch in Python



Revisiting the ReAct Framework

Reasoning and Acting Cycle:

- The agent alternates between thinking (*Thought*) and doing (*Action*).
- Observes the outcome (*Observation*) and repeats the cycle until the task is complete.



The ReAct Workflow



Crafting Effective Prompts

- **Importance of Prompts:**

- Direct the agent's reasoning and actions.
- Define how the agent should think and respond.

- **Prompt Structure:**

- Instructions for the agent's behavior.
 - **Example:** *You run in a loop of Thought, Action, PAUSE, Observation. At the end of the loop you output an Answer.*
- Definition of available actions.
 - **Example:** *Your available actions are:*
calculate_total_price: ...
get_fruit_price: ...



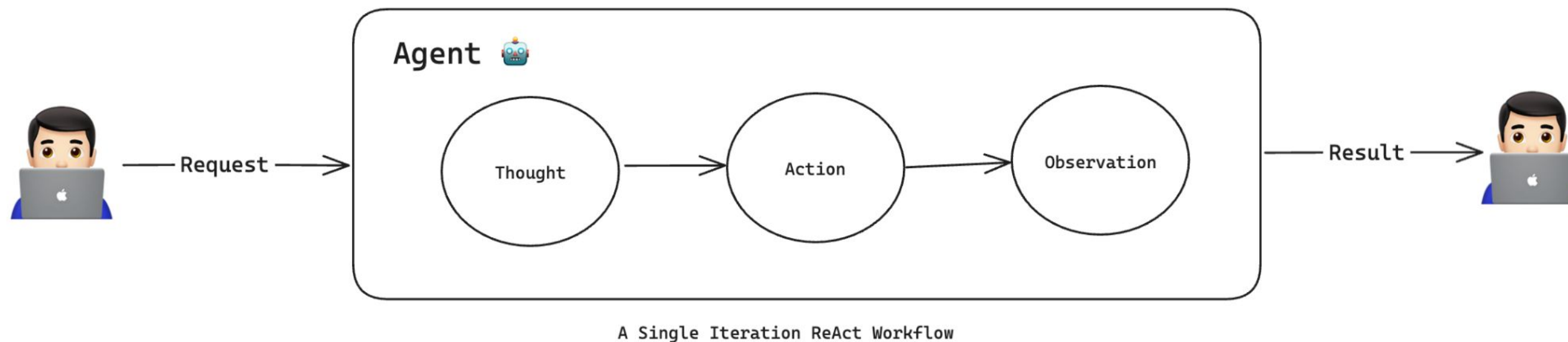
Implementation Plan

1. Define the agent class
 - a. Attributes: System prompt, message history, available actions
 - b. Methods: Handling messages, executing Actions
2. Craft prompts
3. Add tools
4. Add query handling capabilities
5. Add loops to handle complex queries (later)

Implementing the Agent Without Loops



- **Workflow Without Loops:**
 - The agent performs a single *Thought* → *Action* → *Observation* flow.
- **Limitations:**
 - Cannot handle tasks that require multiple reasoning steps.



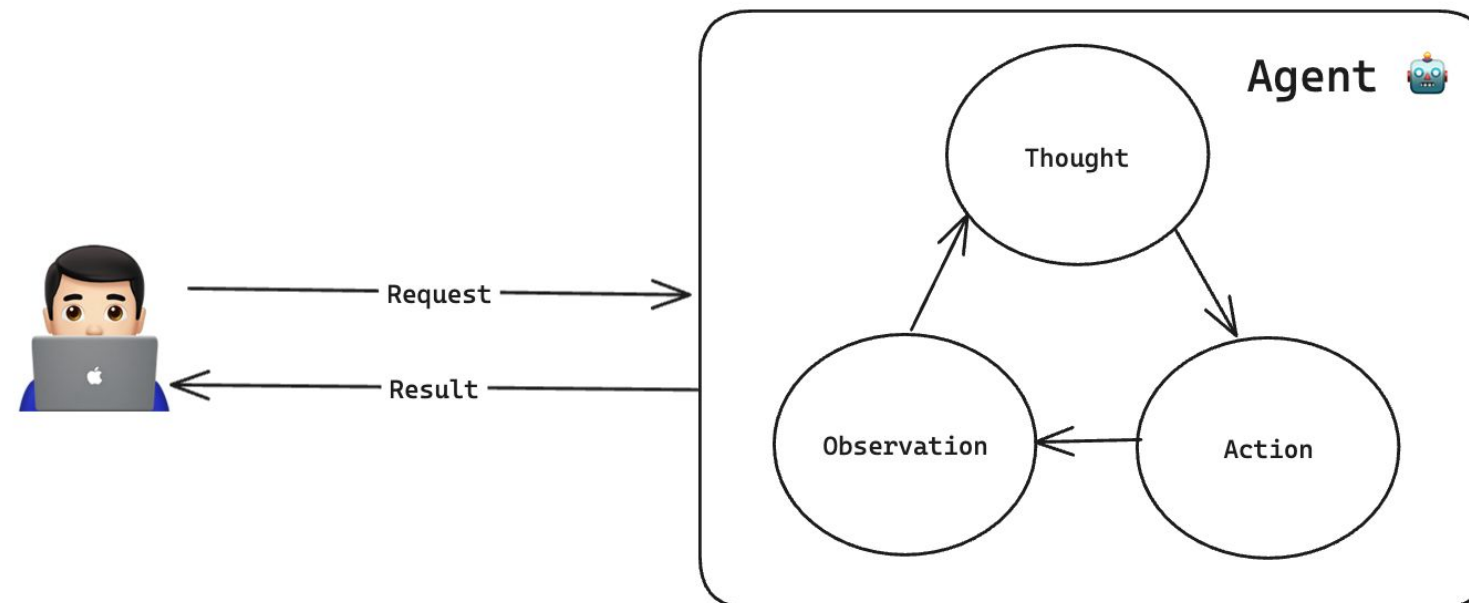
O'REILLY®

2.6 Implementing Loops for Multi-Step Agent Tasks



Enhancing the Agent with Cycles

- **Introducing Loops for Complex Tasks:**
 - Allows the agent to perform iterative reasoning and actions.
- **Benefits:**
 - Handles multi-step queries.
 - Improves the agent's problem-solving abilities.



The ReAct Workflow with Cycles

O'REILLY®

2.7 Understanding Nodes and Edges in LangGraph





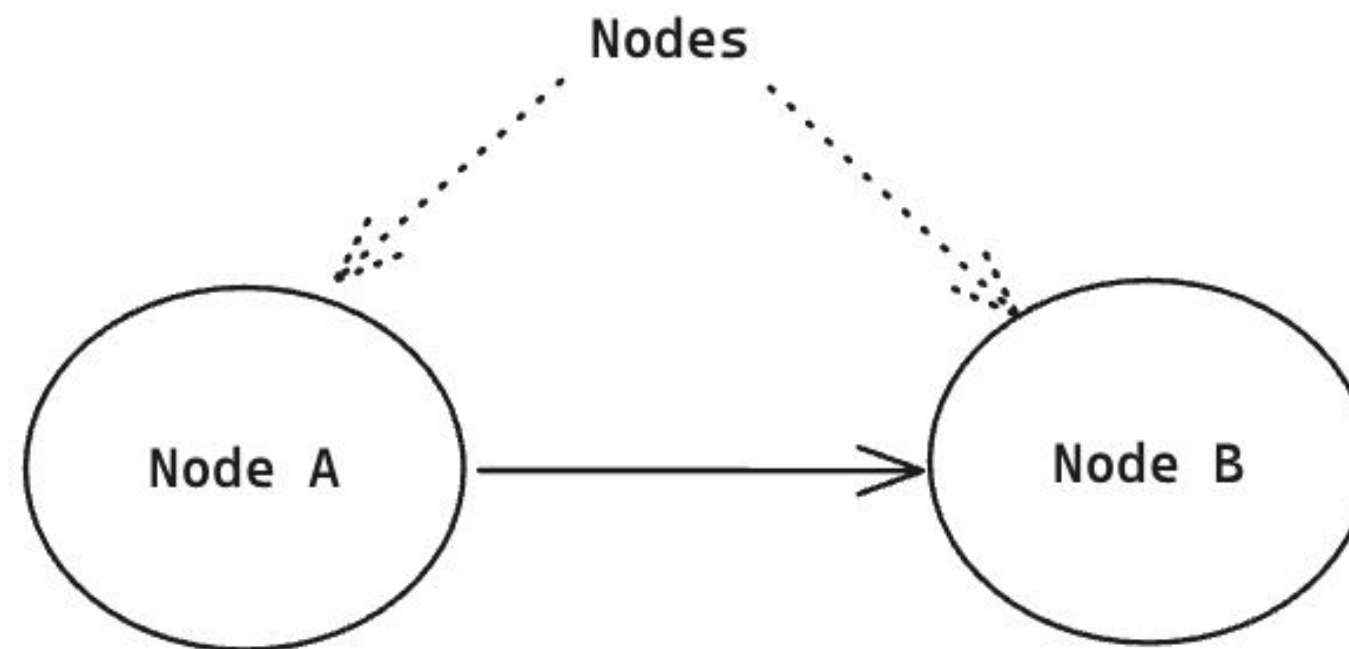
Agent Workflows as Graphs

- **Graph-Based Modelling in LangGraph**
 - LangGraph represents agent workflows as graphs.
 - Nodes and Edges depict the flow of control and data.
- **Key Components of a Graph in LangGraph**
 - State
 - Nodes
 - Edges



Nodes in LangGraph

- Nodes are functions that **encode the logic of an agent**.
- Receive the current state as input, perform computations, and return an updated state.
- Can include LLM calls or any other Python code.



Example of Nodes



Implementing Nodes

- Nodes are added to the graph using the `add\_node` method.
- The first argument to a node is the state.
- A second, optional argument is the "config" for additional parameters, ex: thread_id.

```
from langchain_core.runnables import RunnableConfig
```

```
from langgraph.graph import StateGraph
```

```
builder = StateGraph(dict)
```

```
def my_node(state: dict, config: RunnableConfig):
```

```
    return {"results": f"Hello, {state['input']}!"}
```

```
# The second argument is optional
```

```
def my_other_node(state: dict):
```

```
    return state
```

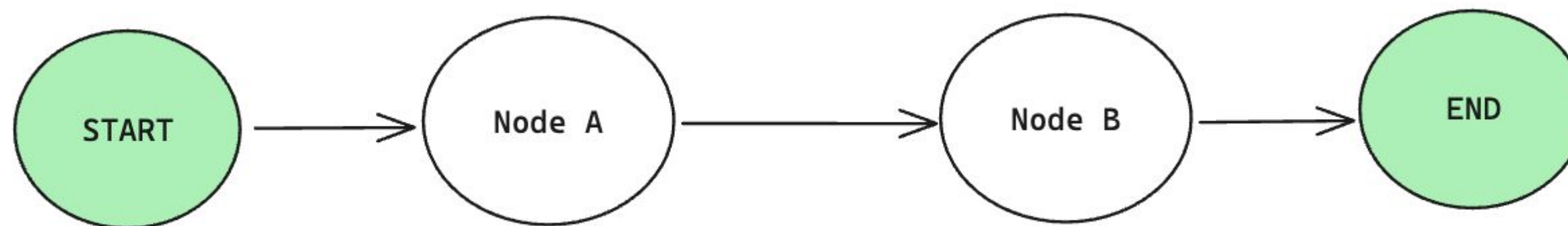
```
builder.add_node("my_node", my_node)
```

```
builder.add_node("other_node", my_other_node)
```



Special Nodes

- **START Node**
 - Entry point of the graph where user input is received.
- **END Node**
 - Terminal point of the graph indicating completion.

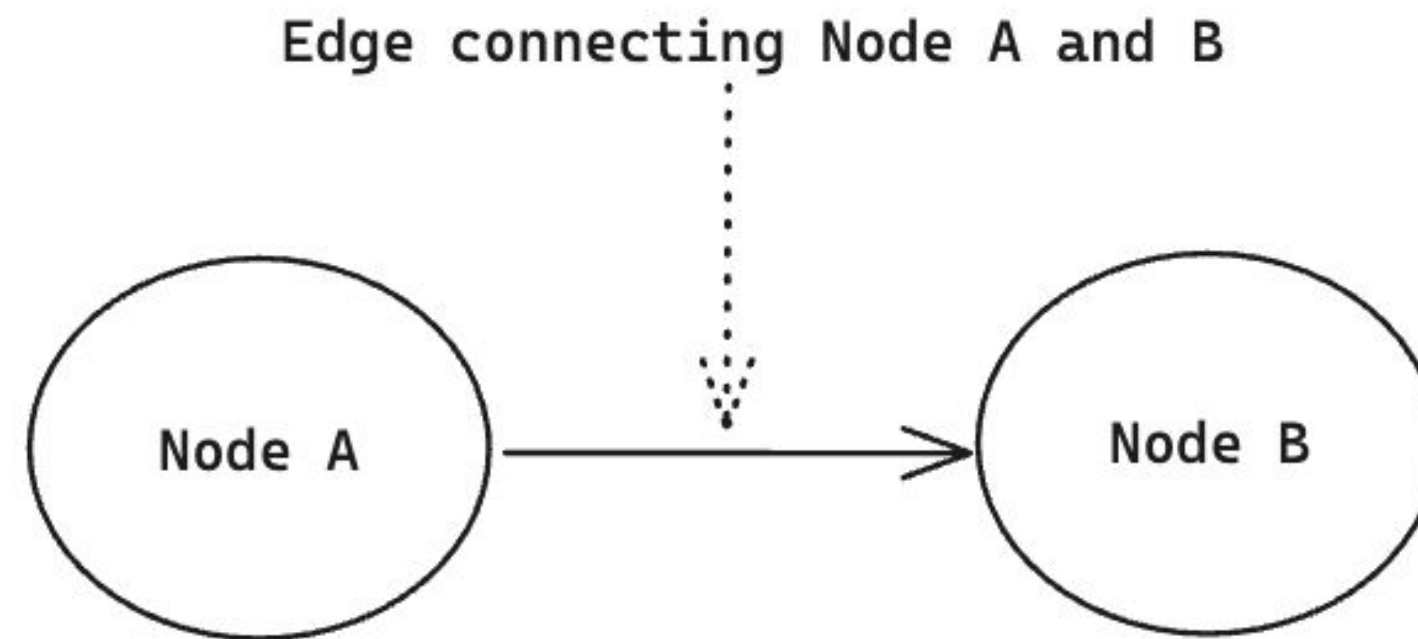


The START and END Special nodes



Edges in LangGraph

- Functions that determine which node to execute next based on the current state.
- Define the logic flow between nodes.



An Example Edge



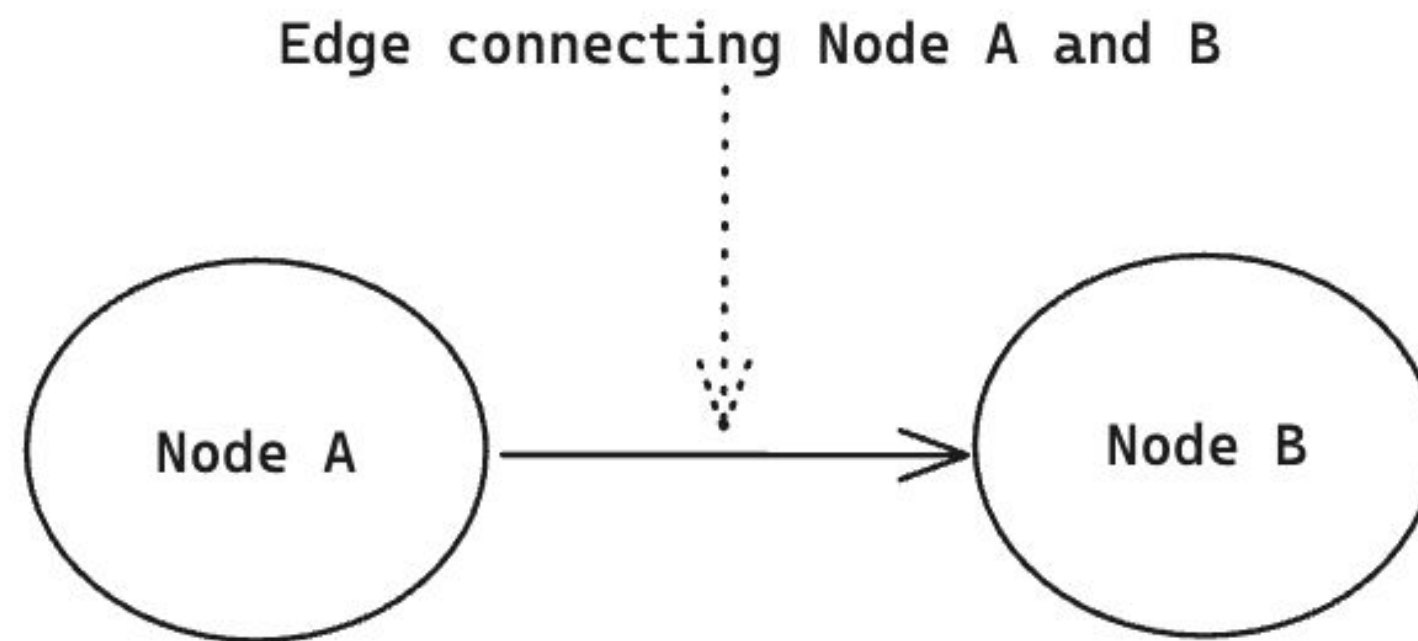
Types of Edges

- **Normal Edges**
 - Direct transitions from one node to the next.
- **Conditional Edges**
 - Route to different nodes based on evaluation of the state.
- **Conditional Entry Points**
 - Allow the graph to start at different nodes based on custom logic.



Normal Edges

```
builder.add_edge("node_a", "node_b")
```

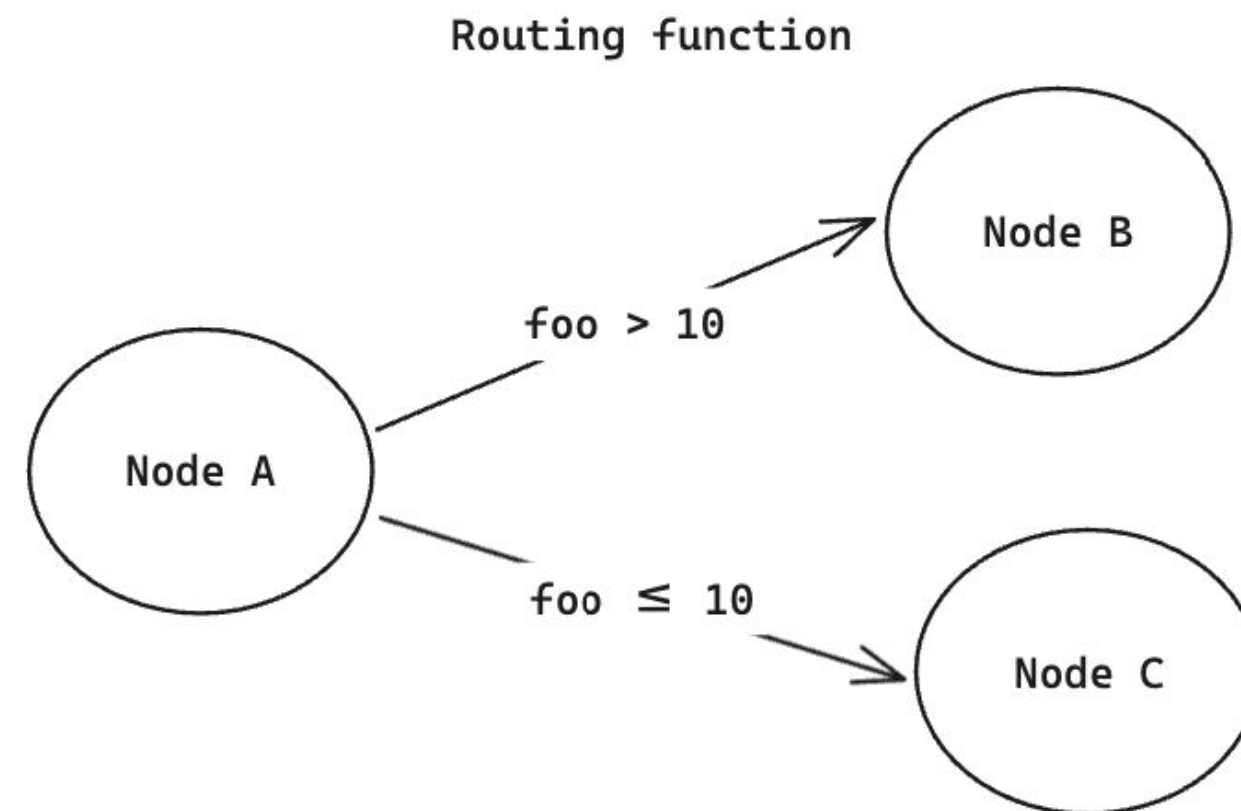


An Example Edge



Conditional Edges

```
def routing_function(state):  
    return state["foo"] > 10 # return True or False based on state  
  
graph.add_conditional_edges("node_a", routing_function, {True: "node_b", False: "node_c"})
```

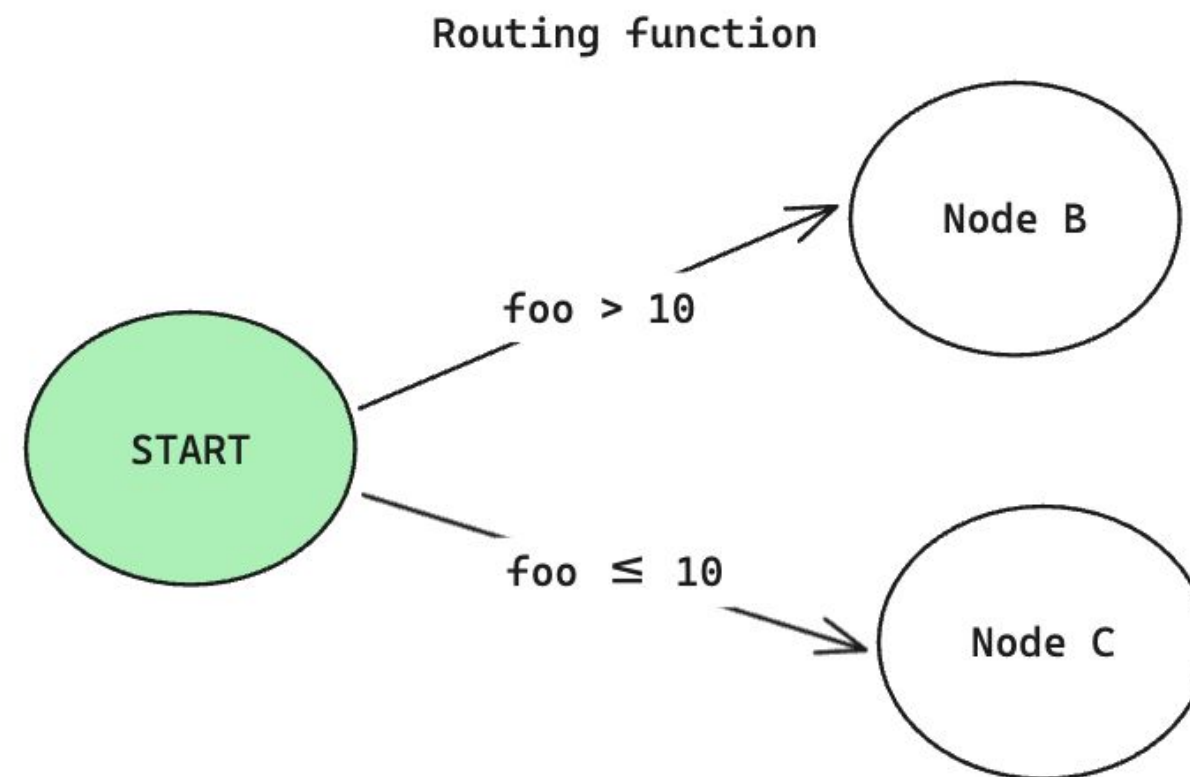


An Example Conditional Edge



Conditional Entry Point

```
def routing_function(state):  
    return state["foo"] > 10 # return True or False based on state  
  
graph.add_conditional_edges(START, routing_function, {True: "node_b", False: "node_c"})
```



An Example Conditional Starting Point

O'REILLY®

2.8 Defining and Managing State in LangGraph





Understanding State

- **What is State?**
 - A shared data structure representing the current snapshot of the agentic system.
 - Evolves over time as nodes perform computations and pass data.
- **Role of State**
 - Serves as the input and output schema for all nodes and edges.
 - Ensures consistency in data flow and throughout the graph.



Understanding State

Example:

```
from typing import TypedDict
```

```
class State(TypedDict):
```

```
    foo: int
```

```
    bar: list[str]
```



State Schema and Reducers

- **State Schema**
 - Defines the structure of the state.
 - Using **TypedDict**: Sketches a type-checked dictionary structure.
 - Using **Pydantic BaseModel**: Enriches the schema by adding validation and default values.
- **Reducers**
 - Determine how updates are applied to the state.
 - Each key in the state has its own independent reducer function



Types of Reducers

Default Reducer: Updates from nodes override existing state values.

```
from typing import TypedDict
```

```
class State(TypedDict):
```

```
    foo: int
```

```
    bar: list[str]
```

Annotated Reducer: Use custom reducer functions like `operator.add` to combine values.

```
from typing import TypedDict, Annotated
```

```
from operator import add
```

```
class State(TypedDict):
```

```
    foo: int
```

```
    bar: Annotated[list[str], add]
```



Types of Reducers

Default Reducer Example:

- **Initial State:** `{"foo": 1, "bar": ["hi"]}`
- **Node Update:** `{"foo": 2}`
- **Updated State:** `{"foo": 2, "bar": ["hi"]}`
- **Explanation:**
 - The value of `foo` is overridden.

Annotated Reducer Example:

- **Reducer Annotation:** `bar: Reducer(operator.add)`
- **Node Update:** `{"bar": ["bye"]}`
- **Updated State:** `{"foo": 1, "bar": ["hi", "bye"]}`
- **Explanation:**
 - The lists are combined using `operator.add`.

O'REILLY®

2.9 Debugging Agent Executions with Logging



Why Debugging is Critical

- **The Challenge: "What Went Wrong?"**
 - Agent fails to complete task → hard to diagnose
 - Non-deterministic LLM outputs
 - Multi-step workflows with hidden reasoning
 - Multiple failure points: LLM, tools, APIs, state
- **The Solution: Comprehensive Logging**
 - Track every decision point in the agent loop
 - Capture inputs, outputs, and intermediate steps
 - Transform debugging from guesswork to analysis

What Needs to be Logged

- **LLM Interactions**
 - Inputs (prompts) and outputs (responses)
 - Token usage and latency
- **Tool Activity**
 - Tool call proposals and arguments
 - Tool execution results and observations
- **Workflow State**
 - State transitions across workflow
 - Errors and exceptions

Sample Pseudocode for Logging



```
# Inside agent loop
while not done:
    logger.info(f"Turn {turn}")

    response = llm.invoke(messages)
    logger.info(f"LLM response: {response.content[:100]}...")

    if response.tool_calls:
        logger.info(f"Tool: {tool_name}({tool_args})")
        result = execute_tool(tool_name, tool_args)
        logger.info(f"Result: {result}")
```


Structured Logging for Production Agents

- **Why Structured Logging?**
 - JSON format for machine parsing
 - Easy integration with observability platforms
 - Searchable, filterable logs
- **Key Metadata to Include**
 - Timestamps and request IDs
 - Model name and parameters
 - Token counts and latency
 - Tool names and arguments

Structured Logging in Code

```
import json
from datetime import datetime

def log_event(event_type, data):
    log_entry = {
        "timestamp": datetime.utcnow().isoformat(),
        "event": event_type,
        "data": data
    }
    logger.info(json.dumps(log_entry))

# Usage in agent nodes
def agent_node(state):
    log_event("llm_call_start", {"messages":
state["messages"]})
    response = model.invoke(state["messages"])
    log_event("llm_call_complete", {
        "content": response.content,
        "tool_calls": response.tool_calls,
        "tokens": response.usage_metadata
    })
```

Logging in Observability Platforms

- **What Platforms Provide**
 - Visual trace trees showing entire agent run
 - Automatic token and cost tracking
 - Comparison of runs across versions
 - Evaluation metrics and feedback loops
- **Leading Tools (2025)**
 - LangSmith, Langfuse, OpenTelemetry, Portkey, etc

Logging Best Practices

- **Start Logging Early**
 - Implement observability from day one
 - Easier to debug during development
 - Prevents production surprises
- **Balance Detail vs. Cost**
 - Logging full prompts/responses in production is expensive
 - Consider truncation or summarization strategies, only log the necessary parts
 - Store full traces only for failed runs

Logging Best Practices

- **Log at Appropriate Levels**
 - **DEBUG:** Detailed diagnostics (state, intermediate values)
 - **INFO:** Key events (LLM calls, tool use, completions)
 - **WARNING:** Unexpected but handled situations
 - **ERROR:** Failures requiring attention



Introduction to LangSmith

- **What is LangSmith?**
 - An all-in-one developer platform designed to support every step of the LLM-powered application lifecycle.
 - Facilitates debugging, collaboration, testing, and monitoring of LangGraph and LangChain applications.
- **Why Use LangSmith?**
 - **Enhanced Visibility:** Gain comprehensive insights into agent workflows and performance.
 - **Real-Time Debugging:** Identify and resolve issues promptly with detailed trace data.
 - **Collaboration:** Share trace data and collaborate with team members to improve agent reliability.

O'REILLY®

2.10 Troubleshooting Common LLM API



Overview of Common LLM API Errors

- Authentication Errors (401, 403)
- Rate Limiting Errors (429)
- Service Availability Errors (500, 502, 503)

Authentication Errors

- Missing or invalid API keys
- Incorrect key format (OpenAI: sk-, Anthropic: sk-ant-)
- Expired credentials or suspended account
- Wrong authorization headers

Quick Validation and Detection



```
# Check key is set
api_key = os.getenv("OPENAI_API_KEY")
if not api_key or not api_key.startswith("sk-"):
    raise ValueError("Invalid or missing API key")

# Error Detection
except Exception as e:
    if "401" in str(e):
        # Check API key configuration
    elif "403" in str(e):
        # Check account status and permissions
```

Rate Limiting Errors

- Common Types of Rate Limits when using LLMS
 - Requests per minute (RPM)
 - Tokens per minute (TPM)
 - Tokens per day (TPD)
 - Concurrent request limits

Mitigation: Exponential Backoff



```
from openai import RateLimitError

for attempt in range(5): # Max 5 retries
    try:
        response = client.chat.completions.create(...)
        break # Success, exit loop
    except RateLimitError:
        wait_time = 2 ** attempt # 1, 2, 4, 8, 16 seconds
        time.sleep(wait_time)
else:
    raise Exception("Max retries exceeded")
```

Mitigation: Using Retry Libraries



```
from tenacity import retry, wait_exponential

@retry(wait=wait_exponential(min=2, max=60))
def call_llm(messages):
    return client.chat.completions.create(model="gpt-5",
messages=messages)
```

Prevention Strategies

- Implement request queuing to smooth out bursts
- Batch requests where the API supports it
- Request higher rate limits from your provider
- Use multiple providers as fallbacks (not to circumvent limits)

Service Availability Errors

- **Common Types of Availability Errors**
 - 500 Internal Server Error: API-side bug
 - 502 Bad Gateway: Proxy/load balancer issue
 - 503 Service Unavailable: Maintenance or overload
 - Scheduled maintenance windows

Service Availability Mitigation Strategies

- Implement circuit breakers for repeated failures
- Define fallback models/providers
- Cache responses when appropriate
- Subscribe to status page notifications

Example: Fallback Strategy



```
try:
    response = client.chat.completions.create(model="gpt-5",
    ...)
except APIError as e:
    if e.status_code >= 500:
        # Fallback to alternative model
        response = client.chat.completions.create(model="gpt-
4o", ...)
```

Example: Monitor Service Status



```
# Monitor provider status pages
status_url = "https://status.openai.com/api/v2/status.json"
response = requests.get(status_url)
status = response.json()["status"]["indicator"] # "none",
"minor", "major"
```

Network and Timeout Errors

- **Common Network Issues**

- Connection timeouts (slow networks)
- Read timeouts (slow API responses)
- DNS resolution failures
- Proxy/firewall blocking

- **Timeout Configuration**

- Most LLM client SDKs support timeout and max_retries parameters
- Can also implement custom timeout logic with Python's `asyncio.timeout` or `httpx` client settings

O'REILLY®

2.11 Building a Chatbot Agent in LangGraph

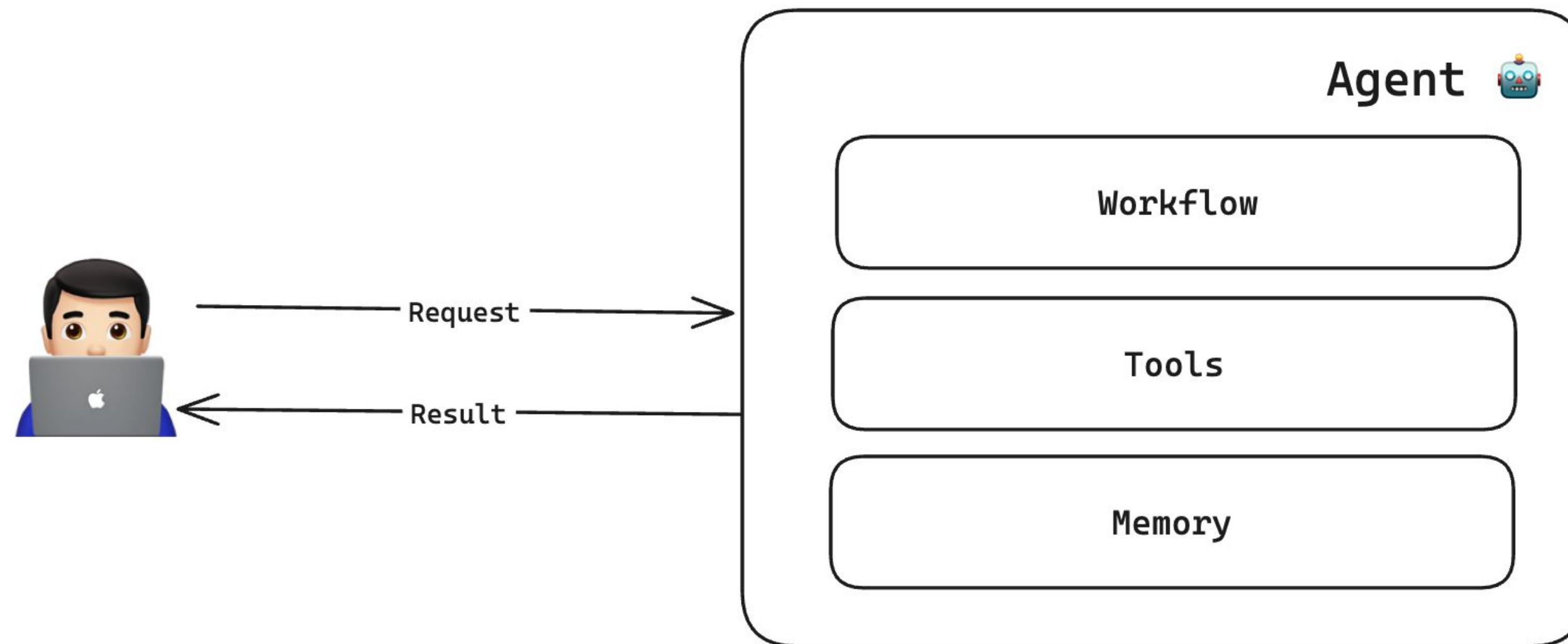


Building a Chatbot Agent in LangGraph



- **Objective:**
 - Build a basic chatbot agent using LangGraph.
 - Enhance the chatbot with tool integration and memory capabilities.
- **Overview:**
 - Step 1: Create a basic chat agent.
 - Step 2: Integrate web search tool to extend functionality.
 - Step 3: Implement memory to maintain conversation state.

Building a Chatbot Agent in LangGraph



An Example Chatbot Agent with LangGraph



2.12 Implementing Streaming Output for Real-Time Responses



Understanding Streaming

- **What is Streaming?**

- Delivering tokens as they're generated
- Real-time display instead of waiting for completion
- Similar to ChatGPT's typing effect
- Most frameworks / SDKs will offer streaming option

- **Why Stream?**

- Better user experience
- Perceived lower latency
- Transparency into processing
- Ability to cancel long-running requests

Streaming in LangChain



```
from langchain_openai import ChatOpenAI

model = ChatOpenAI(model="gpt-4o")

# Stream tokens
for chunk in model.stream("Write a short story"):
    print(chunk.content, end="", flush=True)
```

Streaming in LangGraph

- **Stream Modes**
 - "values": Stream state updates
 - "messages": Stream LLM tokens
 - "updates": Stream node outputs

Streaming in LangGraph



```
from langgraph.graph import StateGraph

graph = create_agent_graph()
app = graph.compile()

# Stream messages mode
for chunk in app.stream(
    {"messages": [("user", "Hello")]},
    stream_mode="messages"
):
    message_chunk, metadata = chunk
    print(message_chunk.content, end="", flush=True)
```



2.13 Configuring Async and Sync Agent Execution



Sync vs Async Fundamentals

- **Synchronous Execution**

- Blocking operations
- One task at a time
- Simple to understand and debug

- **Asynchronous Execution**

- Non-blocking operations
- Multiple tasks concurrently
- Better for I/O-bound operations
- Essential for high-performance applications

Async Execution for LLM Applications

- **LLM Applications are I/O-Bound**
 - Most time spent waiting for network responses, not computing
 - CPU sits idle during API calls
- **Common I/O Operations in Agents**
 - API calls to LLM providers (often 1-30+ seconds)
 - Tool execution (web searches, database queries)
- **The Opportunity**
 - While waiting for one LLM response, initiate other requests
 - Run multiple tool calls in parallel
 - Handle multiple users simultaneously

Conceptual Example

- **Consider an I/O-bound operation taking 2 seconds per request**
- **Synchronous Execution**
 - 6 requests executed one after another
 - Total time: $6 \times 2 \text{ seconds} = 12 \text{ seconds}$
- **Asynchronous Execution**
 - 6 requests initiated simultaneously
 - Total time: ~2 seconds (time of slowest request)
 - All requests execute concurrently
- Performance improvement scales with number of concurrent operations

Implementing Async Agents in LangGraph

- **Async Method Naming**
 - **Sync:** `.invoke()`, `.stream()`
 - **Async:** `.ainvoke()`, `.astream()` (prefixed with "a")

```
● ● ●  
  
# .. app defined in code earlier  
# Sync execution  
result = app.invoke({"input": "query"})  
  
# Sync streaming  
for chunk in app.stream({"input":  
    "query"}):  
    print(chunk)
```

```
● ● ●  
  
# .. app defined in code earlier  
# Async execution  
result = await app.ainvoke({"input":  
    "query"})  
  
# Async streaming  
async for chunk in  
    app.astream({"input": "query"}):  
    print(chunk)
```

Mixing Sync and Async Code

- **The Challenge**
 - Can't directly call async from sync
 - Calling sync in async kills performance

Wrapping Sync Functions

```
import asyncio

def slow_sync_function(x):
    """Legacy sync function"""
    import time
    time.sleep(2)
    return x * 2

# WRONG: Blocks event loop
async def bad_async_node(state):
    result = slow_sync_function(state["value"]) # Blocks!
    return {"value": result}

# CORRECT: Use to_thread
async def good_async_node(state):
    result = await asyncio.to_thread(
        slow_sync_function,
        state["value"]
    )
    return {"value": result}
```

Running Async from Sync (if needed!)



```
import asyncio

async def async_agent_call():
    return await agent.ainvoke({"input": "query"})

# In sync context
result = asyncio.run(async_agent_call())
```

When to Use Sync vs Async

- **Use Sync When**

- Simple, single-user scripts
- Sequential workflows without I/O
- Learning and prototyping
- Debugging complex issues
- CPU-bound operations

- **Use Async When**

- Building chatbots or web services
- Multiple concurrent users
- High-throughput requirements
- Many I/O operations (API calls, DB queries)
- Production deployments
- Real-time streaming needed

O'REILLY®

2.14 Creating Reusable Dynamic Prompt Templates



Why Prompt Templates?

- **Benefits of Templates**
 - Reusability across different use cases
 - Consistency in prompt structure
 - Easy maintenance and updates
 - Dynamic content population
 - Separation of prompt logic from code

Why Prompt Templates?

- **Problems They Solve**
 - Hardcoded prompts are inflexible
 - Difficult to modify at scale
 - No way to customize for different inputs
 - Hard to version and test

Use Cases for Prompt Template

- **Multi-user applications**
 - Same prompt structure, different user data
 - A/B testing different prompts
 - Swap templates without code changes
- **Localization and personalization**
 - Language-specific or user-specific variations
- **Domain-specific variations**
 - Tailored prompts for different industries or contexts

Prompt Templates in LangChain

- Designed for single message interaction with LLMs
- Uses curly braces for variables
- Simple string substitution

Prompt Template Example

```
from langchain_core.prompts import PromptTemplate

# Define template
template = PromptTemplate.from_template(
    "Tell me a {adjective} joke about {topic}"
)

# Use template
prompt = template.invoke({
    "adjective": "funny",
    "topic": "programming"
})

# Output: "Tell me a funny joke about programming"
```

Chat Prompt Templates in LangChain

- Designed for multi-message interactions with LLMs
- Formats lists of messages
- Supports system, user, and assistant roles

Chat Prompt Template Example



```
from langchain_core.prompts import ChatPromptTemplate

template = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful {role}"),
    ("user", "{input}")
])

messages = template.invoke({
    "role": "coding assistant",
    "input": "How do I reverse a string?"
})
```

Common Pitfall: Templates without Version Control

Prompt templates change frequently during development.

Without version control, you can't reproduce past results or roll back breaking changes.

Treat prompts as code: version them, review changes, and track what's deployed.

Tools such as Langfuse, LangSmith offer built-in prompt template management, versioning, tagging etc.

O'REILLY®

2.17 Implementing Input Validation and Guardrails



What are LLM Guardrails?

- **Definition**

- Protective measures for LLM safety and reliability, applicable to inputs to LLMs and outputs from LLMs

- **Why do we need guardrails?**

- Prevent unsafe or harmful outputs
- Protect sensitive information
- Ensure policy compliance
- Maintain system reliability

What are LLM Guardrails?

- **Types of Guardrails**
 - **Input Guardrails:** Validate before processing
 - **Output Guardrails:** Filter generated content
 - **Runtime Guardrails:** Monitor during execution

Inputs to Validate

- Format and structure (length, encoding, data types)
- Content appropriateness (language, tone, topic)
- Injection attack attempts (malicious instructions)
- PII and sensitive data (emails, SSN, credentials)
- Query relevance (on-topic, within scope)

Input Validation Approaches

- **Rule-Based:** Pattern matching, regex, length checks
- **Statistical:** Anomaly detection, frequency analysis
- **LLM-Based:** Semantic understanding, context awareness
- **Key Principle: Fail fast and fail safe**
 - Validate early in the pipeline
 - Provide clear error messages
 - Default to rejection when uncertain

Guardrail Implementation Patterns

- **Single-Stage Guardrails**

- One validation step before processing
- Simple, fast, low-cost
- Best for: Clear rules, binary decisions

- **Multi-Stage Guardrails**

- Sequential validation layers
- Each layer catches different issues
- Example: Format → Content → Security → Relevance

- **LLM-as-Judge Pattern**

- Use separate LLM to evaluate safety
- Semantic understanding of threats
- Trade-off: Higher latency and cost

Guardrails in LangGraph

- **Validation as Graph Nodes**

- Dedicated nodes for input/output validation
- State tracks validation results and errors
- Conditional edges route based on validation

- **Placement Strategies**

- **Entry Point:** Validate before any processing
- **Pre-Agent:** Check after retrieval, before reasoning
- **Pre-Tool:** Validate before tool execution
- **Exit Point:** Final output validation

Layered Defense Strategy

- **Layer 1:** Fast rule-based checks (regex, length, format)
- **Layer 2:** Statistical analysis (anomaly detection, patterns)
- **Layer 3:** LLM-based semantic validation (context, intent)
- Escalate only when needed to optimize cost/latency

Hybrid Validation Patterns

- **Combine deterministic + probabilistic methods**
 - Rules handle predictable cases quickly
 - LLMs catch edge cases rules miss
- **Use rules for known threats, LLMs for novel ones**
 - Blocklists for known bad patterns
 - LLM classifiers for ambiguous content
- **Aggregate multiple signals for confidence scoring**
 - Weigh results from each validation method
 - Set thresholds for accept/reject/review

Prompt Injection Defense

- **Pattern matching for common injection phrases**
 - Detect "ignore previous instructions", "you are now..."
 - Regex rules for known attack patterns
- **Delimiter separation (user vs system instructions)**
 - Clear boundaries between trusted and untrusted content
 - Use XML tags or special tokens as separators

Prompt Injection Defense

- **LLM-based intent classification**
 - Classify input as legitimate vs adversarial
 - Detect subtle manipulation attempts
- **Input sanitization and encoding**
 - Escape special characters
 - Normalize Unicode to prevent obfuscation

O'REILLY®

2.16 Writing Test Cases for Agent Actions



What are Test Cases?

- A test case specifies: given input → expected output
- **Why Test?**
 - Verify tools work correctly in isolation
 - Confirm agent takes the right action
 - Build confidence in your system
- **What to Test**
 - Individual tool functions (e.g., calculator, weather lookup)
 - Single agent actions (e.g., tool selection, response generation)
 - Both success cases and error handling

The Challenge with Testing AI Agents

- **Traditional software**
 - Deterministic inputs → deterministic outputs
- **AI Agents:**
 - Same input can produce different (but correct) outputs
 - Multi-step reasoning paths may vary between runs
 - External tool calls add complexity

Testing Philosophy for AI Agents

- **Deterministic testing**
 - Verify tool behavior, state management, APIs
- **Stochastic testing**
 - Evaluate LLM output quality using metrics
- **Behavioral testing**
 - Ensure agent makes correct decisions

Anatomy of a Simple Test Case

- **The Three Parts (Arrange-Act-Assert)**
 - **Arrange:** Set up your input data
 - **Act:** Execute the function or agent action
 - **Assert:** Verify the output matches expectations

Example: Testing a Tool

```
import pytest
from langchain_core.tools import tool

@tool
def calculate(expression: str) -> float:
    """Evaluate a mathematical expression"""
    return eval(expression)

def test_calculator_tool():
    # Arrange: Define input
    input_data = {"expression": "2 + 2"}
    # Act: Execute the tool
    result = calculate.invoke(input_data)
    # Assert: Check expected output
    assert result == 4
```

Testing a Single Agent Action

- Was any tool called by the agent?
- Was the tool called with the right parameters?
- Does the final response contain expected information?
- Did the agent complete the task successfully?

Writing Good Test Cases

- **Characteristics of Good Tests**

- **Clear:** Easy to understand what's being tested
- **Focused:** Tests one thing at a time
- **Independent:** Doesn't rely on other tests
- **Repeatable:** Same input always gives same result

- **Test Both Success and Failure**

- **Happy path:** Valid input → correct output
- **Error path:** Invalid input → appropriate error

Common Pitfalls

- **Testing With Perfect Inputs Only**
 - Your test suite uses clean, well-formatted queries. Production users send typos, incomplete sentences, multiple questions in one message, and irrelevant preambles. Include messy, realistic inputs in your test suite.
- **No Regression Testing**
 - You fix a bug for Query A, but break Query B in the process. Without regression tests, you're playing whack-a-mole. Every bug fix should add a test case to prevent recurrence.



2.17 Measuring Agent Performance and Cost



Why Measure Performance?

- Quantify agent effectiveness
- Identify performance bottlenecks
- Optimize costs
- Track improvements over time
- Support production monitoring

Core Metric Categories

- **Latency:** Response time
- **Token Usage:** Input/output tokens
- **Cost:** API expenses
- **Success Rate:** Task completion percentage
- **Quality:** Accuracy and correctness

Latency Metrics

- **What to Measure**

- End-to-end response time: Total time from request to final response
- Time to first token (TTFT): Time until first output token (critical for streaming)
- Per-node processing time: Time spent in each graph node (for workflows)
- LLM call latency: Time taken for model inference
- Tool execution time: Duration of external tool/API calls

- **Percentile Tracking**

- P50 (median): Typical user experience
- P95: Experience of 95% of users
- P99: Worst-case scenarios

Token Usage and Cost Metrics

- **Token Tracking**

- **Input tokens:** Tokens in prompt and context
- **Output tokens:** Tokens generated by model
- **Total tokens:** Sum of input and output
- Cumulative tracking across multi-turn conversations
- Per-session and per-request granularity

- **Cost Calculation Components**

- Input token cost: Based on prompt size
- Output token cost: Based on generation length (typically higher rate)
- Model-specific pricing tiers

Success Rate Tracking

- **Total requests:** Overall volume
- **Successful requests:** Completed without errors
- **Failed requests:** Exceptions or timeouts
- **Success rate calculation:** $(\text{successful} / \text{total}) \times 100$
- **Time-based analysis:** Track trends over periods

Quality Metrics

- **Accuracy:** Percentage of correct answers
- **Task completion:** Percentage without human escalation
- **Error recovery rate:** Percentage of errors handled gracefully
- **Tool selection accuracy:** Percentage of correct tool choices
- **Response relevance:** Quality of generated outputs
- **Hallucination rate:** Frequency of incorrect/fabricated information

Metrics Dashboarding

- **Production Metrics Framework Components**
 - Centralized metrics collection
 - Multi-dimensional tracking (latency, tokens, costs, success)
 - Percentile calculations for latency analysis
 - Aggregated reporting across sessions
 - Historical data retention for trend analysis
- Observability Frameworks such as LangSmith, Langfuse provide turnkey solutions for dashboarding and report generation

O'REILLY®